

Data Structures in R & Python

Lecture 06

Dr. Colin Rundel

Tuples

Tuples

Python tuples are *heterogeneous, ordered, immutable* containers. They are nearly identical to lists except that their contents cannot be changed - you will most often meet them as the return value of functions and in `enumerate()`, `zip()`, and `dict.items()`.

```
1 (1, 2, 3)
```

py

```
(1, 2, 3)
```

```
1 (1, True, "abc")
```

py

```
(1, True, 'abc')
```

```
1 (1, (2, 3), [4, 5])
```

py

```
(1, (2, 3), [4, 5])
```

```
1 type( (1) )
```

py

```
<class 'int'>
```

```
1 type( (1,) )
```

py

```
<class 'tuple'>
```

```
1 x = 1, 2
```

py

```
2 x
```

```
(1, 2)
```

It is the *comma* that creates a tuple, not the parentheses - `(1)` is just the number 1 and a one element tuple must be

Tuples are immutable

Once created, a tuple's elements cannot be replaced or removed. Immutability is *shallow* though - a mutable object stored in a tuple can still be modified in place, which matters for hashing (coming up shortly).

```
1 x = (1, 2, 3)
2 x[2] = 5
```

py

TypeError: 'tuple' object does not support

```
1 del x[2]
```

py

TypeError: 'tuple' object doesn't support

```
1 x = (1, [2, 3])
2 x[1].append(4)
3 x
```

py

(1, [2, 3, 4])

Tuples can be converted to and from lists with `list()` and `tuple()`, and a tuple is never equal to a list with the same

Value unpacking

Assigning a sequence to several names at once *unpacks* it - this is how Python functions return multiple values. The R equivalent is to return a list and extract the pieces.

```
1 def minmax(x):  
2     return min(x), max(x)  
3  
4 minmax([3, 1, 2])
```

py

(1, 3)

```
1 lo, hi = minmax([3, 1, 2])  
2 lo
```

py

1

```
1 hi
```

py

3

```
1 minmax = function(x) {  
2     list(min = min(x), max = max(x))  
3 }  
4  
5 res = minmax(c(3, 1, 2))  
6 str(res)
```

R

List of 2
 \$ min: num 1
 \$ max: num 3

```
1 res$min
```

R

[1] 1

```
1 res[["max"]]
```

R

[1] 3

```
1 x, y = 1, 2
2 x, y = y, x
3 x, y
```

py

(2, 1)

```
1 (x1, y1), (x2, y2) = [[0, 1], [2, 3]]
2 x1, y1
```

py

(0, 1)

```
1 x2, y2
```

py

(2, 3)

Extended unpacking

The number of names must match the length of the sequence unless one name is prefixed with `*`, which collects the remaining values into a list.

```
1 x, y = [1, 2, 3]
```

py

ValueError: too many values to unpack (exp

```
1 first, *rest = range(6)
2 first
```

py

0

```
1 rest
```

py

[1, 2, 3, 4, 5]

```
1 first, *middle, last = range(6)
2 middle
```

py

[1, 2, 3, 4]

```
1 last
```

py

5

```
1 *init, last = "abc"
2 init
```

py

['a', 'b']

Unpacking also happens in `for` loops, which is how `enumerate()`, `zip()`, and `dict.items()` provide several values per iteration,

```
1 pairs = [("a", 1), ("b", 2)]
2 for i, (key, value) in enumerate(pairs):
3     print(i, key, value)
```

py

Dictionaries

Dictionaries

Python `dicts` are *mutable* containers of **key: value** pairs, designed for the efficient lookup of a value by its key. The closest R analog is a named list (or a named atomic vector when the values are homogeneous).

```
1 {"abc": 123, "def": 456}
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict([("abc", 123), ("def", 456)])
```

py

```
{'abc': 123, 'def': 456}
```

```
1 dict(abc=123, hello=456)
```

py

```
{'abc': 123, 'hello': 456}
```

```
1 list(abc = 123, def = 456)
```

R

```
$abc  
[1] 123
```

```
$def  
[1] 456
```

```
1 c(abc = 123, def = 456)
```

R

```
abc def  
123 456
```

Since Python 3.7 dictionaries preserve insertion order. The keyword form of `dict()` only works for keys that are valid

Keys must be hashable

Keys can be any *immutable* (technically, *hashable*) object - numbers, strings, tuples of immutable objects. Values can be anything.

```
1 {1: "abc", 1.1: (1, 1), "one": ["a", "n"], (1, 1): lambda x: x**2}
```

py

```
{1: 'abc', 1.1: (1, 1), 'one': ['a', 'n'], (1, 1): <function <lambda> at 0x10bddbe20>}
```

Using a mutable object as a key is an error, even when it is nested inside a tuple,

```
1 {[1]: "bad"}
```

py

TypeError: unhashable type: 'list'

```
1 {(1, [2]): "bad"}
```

py

TypeError: unhashable type: 'list'

The `hash()` function maps an object to an integer that is used to locate the key in memory - a mutable object's hash could change after insertion, which is why it is disallowed. Instances of user defined classes are hashable by identity

Lookup

`[]` retrieves the value for a key, missing keys are a `KeyError`. Use `in` to test for a key and `.get()` to supply a default.

```
1 x = {1: "abc", "y": "hello", (1, 1): 3.14159}
```

py

```
1 x[1]
```

py

'abc'

```
1 x["y"]
```

py

'hello'

```
1 x[(1, 1)]
```

py

3.14159

```
1 x["def"]
```

py

KeyError: 'def'

```
1 "y" in x
```

py

True

```
1 "hello" in x
```

py

False

```
1 print( x.get("def") )
```

py

None

```
1 x.get("def", 0)
```

py

0

R named lists are more forgiving - a missing name returns `NULL` rather than an error, so check with `%in% names()` or `is.null()`,

```
1 y = list(abc = 123, def = 456)
2 y[["abc"]]
```

R

```
[1] 123
```

```
1 y[["ghi"]]
```

R

```
NULL
```

```
1 "ghi" %in% names(y)
```

R

```
[1] FALSE
```

```
1 is.null(y$ghi)
```

R

```
[1] TRUE
```

Insert, replace, remove

Assigning to a key inserts or replaces, `del` and `.pop()` remove. Note that assigning `None` does *not* remove a key, whereas in R assigning `NULL` does.

```
1 x = {1: "abc", "y": "hello"}
2 x["def"] = -1
3 x["y"] = "goodbye"
4 x
```

py

```
{1: 'abc', 'y': 'goodbye', 'def': -1}
```

```
1 del x[1]
2 x.pop("def")
```

py

```
-1
```

```
1 x
```

py

```
{'y': 'goodbye'}
```

```
1 x["y"] = None
2 x
```

py

```
{'y': None}
```

```
1 y = list(abc = 123, def = 456)
2 y$ghi = -1
3 y$def = "goodbye"
4 str(y)
```

R

```
List of 3
 $ abc: num 123
 $ def: chr "goodbye"
 $ ghi: num -1
```

```
1 y$abc = NULL
2 str(y)
```

R

```
List of 2
 $ def: chr "goodbye"
 $ ghi: num -1
```

```
1 y["def"] = list(NULL)
2 str(y)
```

R

```
List of 2
 $ def: NULL
 $ ghi: num -1
```

Common methods

```
1 x = {1: "abc", "y": "hello"}
```

py

```
1 len(x)
```

py

2

```
1 list(x)
```

py

[1, 'y']

```
1 1 in x
```

py

True

```
1 x.keys()
```

py

dict_keys([1, 'y'])

```
1 x.values()
```

py

dict_values(['abc', 'hello'])

```
1 x.items()
```

py

dict_items([(1, 'abc'), ('y', 'hello')])

```
1 x | {(1, 1): 3.14159}
```

py

{1: 'abc', 'y': 'hello', (1, 1): 3.14159}

```
1 x | {"y": "goodbye"}
```

py

{1: 'abc', 'y': 'goodbye'}

```
1 x.update({"y": "updated", "w": 0})
```

py

```
2 x
```

{1: 'abc', 'y': 'updated', 'w': 0}

`.keys()`, `.values()`, and `.items()` return dynamic views of the dictionary rather than copies - see the [docs](#). `|` returns a

Iterating

Iterating over a `dict` yields its *keys* - use `.items()` to get keys and values together. Comprehensions work too, and `dict(zip(keys, values))` is a common way to build one.

```
1 x = {1: "abc", "y": "hello"}
2 for k in x:
3     print(k)
```

py

1
y

```
1 for k, v in x.items():
2     print(k, "->", v)
```

py

1 -> abc
y -> hello

```
1 y = list(abc = 123, def = 456)
2 for (n in names(y)) {
3     cat(n, "->", y[[n]], "\n")
4 }
```

R

abc -> 123
def -> 456

```
1 names = ["Alice", "Bob", "Carol"]
2 grades = ["A", "A-", "B"]
3
4 {n: g for n, g in zip(names, grades)}
```

py

{'Alice': 'A', 'Bob': 'A-', 'Carol': 'B'}

```
1 dict(zip(names, grades))
```

py

{'Alice': 'A', 'Bob': 'A-', 'Carol': 'B'}

```
1 names = c("Alice", "Bob", "Carol")
2 grades = c("A", "A-", "B")
3 str( setNames(as.list(grades), names) )
```

R

List of 3
\$ Alice: chr "A"
\$ Bob : chr "A-"
\$ Carol: chr "B"

Counting things

The canonical use of a dictionary is tallying - `.get()` with a default avoids checking for the key first. The standard library's `Counter` does this for you, and R has `table()`.

```
1 words = "the quick fox and the lazy dog and"
2
3 counts = {}
4 for w in words:
5     counts[w] = counts.get(w, 0) + 1
6 counts
```

```
{'the': 3, 'quick': 1, 'fox': 1, 'and': 2, 'lazy': 1, 'dog': 1}
```

```
1 from collections import Counter
2 Counter(words).most_common(2)
```

```
[('the', 3), ('and', 2)]
```

```
1 words = strsplit("the quick fox and the lazy dog and")
2 table(words)
```

```
words
and  dog  end  fox  lazy quick  the
  2    1    1    1    1    1    3
```

```
1 sort(table(words), decreasing = TRUE)[1:2]
```

```
words
the and
  3   2
```

`collections.defaultdict(int)` is another option - a missing key is created with a default value (`int()` is `0`) so

Hash maps in R

R has no `dict`. A named list works for small lookup tables but finding a name means scanning all of the names. Environments are R's hash tables - string keys, fast lookup, and (unusually for R) *reference* semantics.

```
1 e = new.env()
2 e$a = 1
3 e[["b"]] = 2
4 assign("c", 3, envir = e)
```

```
1 ls(e)
```

```
[1] "a" "b" "c"
```

```
1 e$a
```

```
[1] 1
```

```
1 exists("b", envir = e)
```

```
[1] TRUE
```

```
1 mget(c("a", "b"), envir = e)
```

```
$a
```

```
[1] 1
```

```
$b
```

```
[1] 2
```

```
1 f = e
2 f$d = 4
3 ls(e)
```

```
[1] "a" "b" "c" "d"
```

```
1 e[1]
```

```
Error in `e[1]`:
! object of type 'environment' is not subs
```

Modifying `f` changed `e` - both names refer to the same environment, exactly like two Python names bound to one `dict`.
`utils::hashtab()` (R 4.2+) provides a hash table with arbitrary keys, and `match()`, `%in%`, `unique()`, and `duplicated()` all

Exercise 1

Represent the JSON record from last lecture in Python using `dicts` and `lists` (JSON happens to be valid Python syntax), then:

```
1 {
2   "firstName": "John",
3   "lastName": "Smith",
4   "age": 25,
5   "address":
6   {
7     "streetAddress": "21 2nd Street",
8     "city": "New York",
9     "state": "NY",
10    "postalCode": 10021
11  },
12  "phoneNumber":
13  [ {
14    "type": "home",
15    "number": "212 555-1239"
16  },
17    {
18    "type": "fax",
19    "number": "646 555-4567"
20  } ]
21 }
```

- Extract the fax number.
- Add a `"mobile"` phone number and remove `age`.
- Write a function `phone(person, type)` that returns the number of the requested type, or `None` if there is no such entry.
- How would `phone()` look in R for the list version from last time?

Sets

Sets

A **set** is a *mutable, unordered* collection of **unique** hashable elements. Sets are constructed with `{}` (no `:`) or `set()`,

```
1 {1, 2, 3, 4, 1, 2}
```

py

```
{1, 2, 3, 4}
```

```
1 set((1, 2, 3, 4, 1, 2))
```

py

```
{1, 2, 3, 4}
```

```
1 set("mississippi")
```

py

```
{'p', 'i', 'm', 's'}
```

```
1 {1, 2, [1, 2]}
```

py

```
TypeError: unhashable type: 'list'
```

```
1 type( {} )
```

py

```
<class 'dict'>
```

```
1 set()
```

py

```
set()
```

Sets are not sequences

Sets have no positions, so `[]` does not work - the primary operation is membership testing with `in`, which is fast regardless of the size of the set.

```
1 x = set(range(5))
```

py

```
1 x[4]
```

py

TypeError: 'set' object is not subscriptable

```
1 3 in x
```

py

True

```
1 len(x)
```

py

5

```
1 x.add(9)
2 x.remove(9)
3 x.discard(8)
4 x.update([7, 8])
5 x
```

py

{0, 1, 2, 3, 4, 7, 8}

```
1 x.remove(6)
```

py

KeyError: 6

`remove()` raises a `KeyError` if the element is absent while `discard()` does not. `pop()` removes and returns an arbitrary

Set operations

Sets support the usual mathematical operations, either as operators (both sides must be sets) or methods (which accept any iterable).

```
1 x = set(range(5))
2 y = set(range(3, 8))
```

py

```
1 x | y
```

py

{0, 1, 2, 3, 4, 5, 6, 7}

```
1 x & y
```

py

{3, 4}

```
1 x - y
```

py

{0, 1, 2}

```
1 x ^ y
```

py

{0, 1, 2, 5, 6, 7}

```
1 x.union(range(3, 8))
```

py

{0, 1, 2, 3, 4, 5, 6, 7}

```
1 x.intersection([3, 4, 9])
```

py

{3, 4}

```
1 x <= set(range(6))
```

py

True

```
1 x.isdisjoint({7, 8})
```

py

True

Sets in R

R has no set type - any vector can be treated as a set using `unique()` and the set functions, all of which return a vector of unique values in order of first appearance.

```
1 a = {"a", "b", "c"}
2 b = {"b", "c", "d"}
3 a | b
```

py

```
{'c', 'a', 'b', 'd'}
```

```
1 a & b
```

py

```
{'b', 'c'}
```

```
1 a - b
```

py

```
{'a'}
```

```
1 "a" in a
```

py

```
True
```

```
1 a = c("a", "b", "c", "a")
2 b = c("b", "c", "d")
3 union(a, b)
```

R

```
[1] "a" "b" "c" "d"
```

```
1 intersect(a, b)
```

R

```
[1] "b" "c"
```

```
1 setdiff(a, b)
```

R

```
[1] "a"
```

```
1 a %in% b
```

R

```
[1] FALSE TRUE TRUE FALSE
```



```
1 words = ["apple", "paper", "pale"]  
2 sorted( set("".join(words)) - {"a"} )
```

py

```
['e', 'l', 'p', 'r']
```

```
1 words = c("apple", "paper", "pale")  
2 setdiff( unique(unlist(strsplit(words,
```

R

```
[1] "p" "l" "e" "r"
```

```
1 duplicated(a)
```

R

```
[1] FALSE FALSE FALSE TRUE
```

Set & dict comprehensions

Comprehensions exist for sets and dictionaries, but not for tuples - parentheses produce a *generator* (more on these later).

```
1 {c.lower() for c in "The Quick Brown"  
{'c', 'o', 'e', 'x', 'u', ' ', 'b', 'r', 'i'}
```

```
1 {n: len(n) for n in ["Alice", "Bob"]}  
{'Alice': 5, 'Bob': 3}
```

```
1 (x**2 for x in range(5))  
<generator object <genexpr> at 0x10be16a80>
```

```
1 tuple(x**2 for x in range(5))  
(0, 1, 4, 9, 16)
```

```
1 {frozenset({1, 2}): "ok"}  
{frozenset({1, 2}): 'ok'}
```

A `frozenset` is an immutable (and therefore hashable) set, which allows a set to be used as a dictionary key or as an

Mutability & copies

Mutable vs immutable

Immutable

`int, float, bool, complex`

`str, bytes`

`tuple, frozenset, range`

`None`

Mutable

`list`

`dict`

`set`

instances of your own classes

Only immutable objects can be dictionary keys or set elements, and mutable objects should never be used as default argument values,

```
1 def f(x=[]):  
2     x.append(1)  
3     return x  
4  
5 f()
```

py

[1]

```
1 f()
```

py

[1, 1]

```
1 def f(x=None):  
2     if x is None:  
3         x = []  
4     x.append(1)  
5     return x  
6  
7 f()
```

py

[1]

```
1 f()
```

py

[1]

Nested containers

In Lecture 2 we saw that `.copy()` is *shallow*, and `copy.deepcopy()` copies recursively. The same sharing problem occurs when building nested structures by repetition,

```
1 grid = [[0] * 3] * 3
2 grid[0][0] = 1
3 grid
```

py

```
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

```
1 grid = [[0] * 3 for _ in range(3)]
2 grid[0][0] = 1
3 grid
```

py

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
1 import copy
2
3 x = {"a": [1, 2], "b": [3, 4]}
4 y = x.copy()
5 z = copy.deepcopy(x)
6
7 x["a"].append(99)
8 y
```

py

```
{'a': [1, 2, 99], 'b': [3, 4]}
```

```
1 z
```

py

```
{'a': [1, 2], 'b': [3, 4]}
```

`[[0] * 3] * 3` creates *one* inner list and references it three times. The comprehension evaluates `[0] * 3` three separate

Copies in R

R copies on modify at every level of a nested structure, so none of this applies - the exceptions are environments (previous section) and objects from packages that wrap external code (e.g. [reticulate](#), [R6](#), database connections).

```
1 x = list(a = list(b = 1))
2 y = x
3 y$a$b = 100
4 x$a$b
```

R

```
[1] 1
```

```
1 y$a$b
```

R

```
[1] 100
```

```
1 x = {"a": {"b": 1}}
2 y = x
3 y["a"]["b"] = 100
4 x["a"]["b"]
```

py

```
100
```

```
1 y["a"]["b"]
```

py

```
100
```

The cost of this convenience is that R has to copy - it does so lazily (only when an object is actually modified and

deques

deques

A [deque](#) (double ended queue) is a list-like container from the [collections](#) module that supports efficient additions and removals at *both* ends. A [list](#) is efficient only at the end.

```
1 from collections import deque
2
3 x = deque(range(3))
4 x
```

```
deque([0, 1, 2])
```

```
1 x.appendleft(-1)
2 x.append(3)
3 x
```

```
deque([-1, 0, 1, 2, 3])
```

```
1 x.popleft()
```

```
-1
```

```
1 x.pop()
```

```
3
```

```
1 x
```

```
deque([0, 1, 2])
```

Use a [deque](#) when you need a queue (first in, first out) - [list.pop\(0\)](#) and [list.insert\(0, v\)](#) have to shift every

maxlen

A `deque` can be given a maximum length - when full, adding to one end drops an element from the other. This makes a rolling window of the last n items trivial.

```
1 x = deque(range(3), maxlen=4)
2 x
```

py

```
deque([0, 1, 2], maxlen=4)
```

```
1 x.append(10)
2 x
```

py

```
deque([0, 1, 2, 10], maxlen=4)
```

```
1 x.append(11)
2 x
```

py

```
deque([1, 2, 10, 11], maxlen=4)
```

```
1 x.appendleft(-1)
2 x
```

py

```
deque([-1, 1, 2, 10], maxlen=4)
```

Algorithms & data structure basics

Big-O notation

Big-O notation describes the *complexity* of an algorithm (usually time, sometimes memory) - how the resources required grow as the size of the input, n , grows.

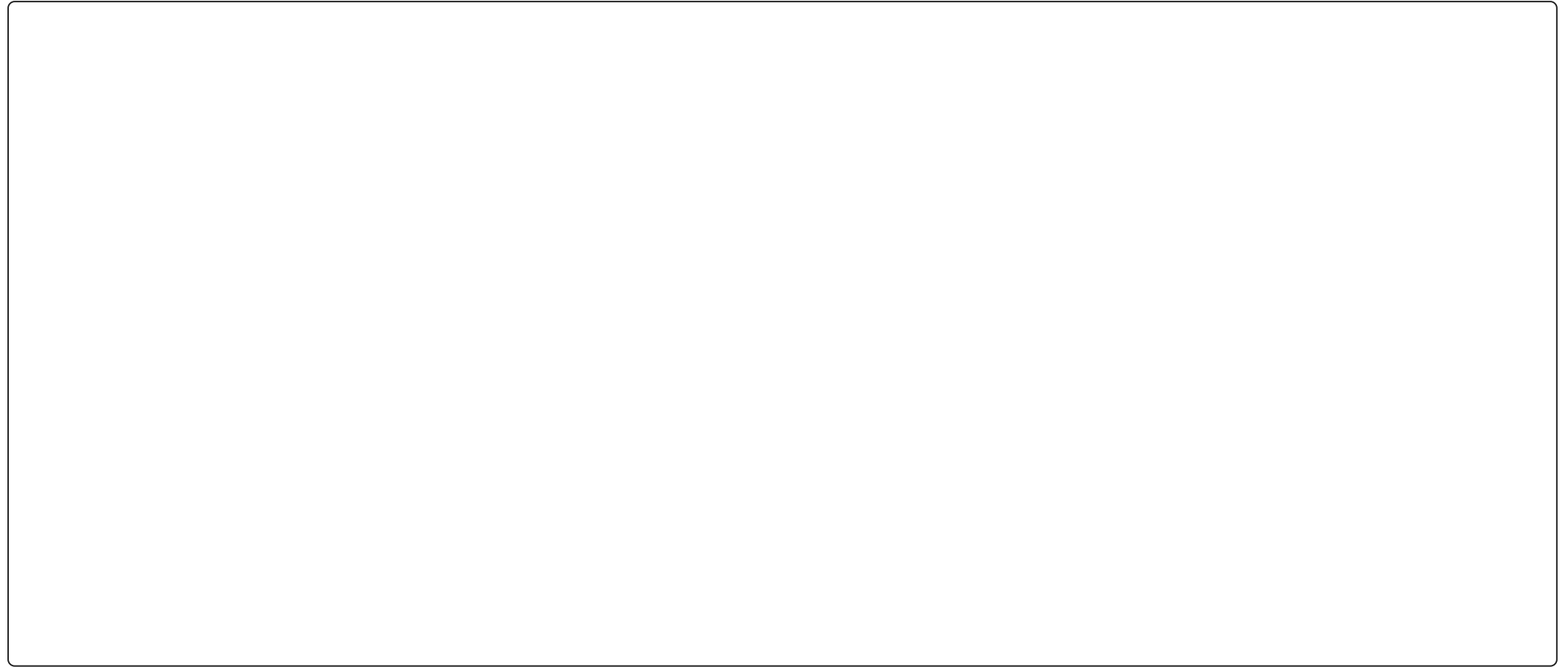
If a function exactly captures the number of steps for an input of size n , the Big-O value is its largest term in n , ignoring constant factors and lower order terms. Two algorithms with the same Big-O can still differ greatly in practice.

Because performance depends on the exact data, we usually talk about *average* and *worst case* complexity, and sometimes *amortized* complexity when an occasional expensive step is paid for by many cheap ones (e.g. growing a list).

Common terminology

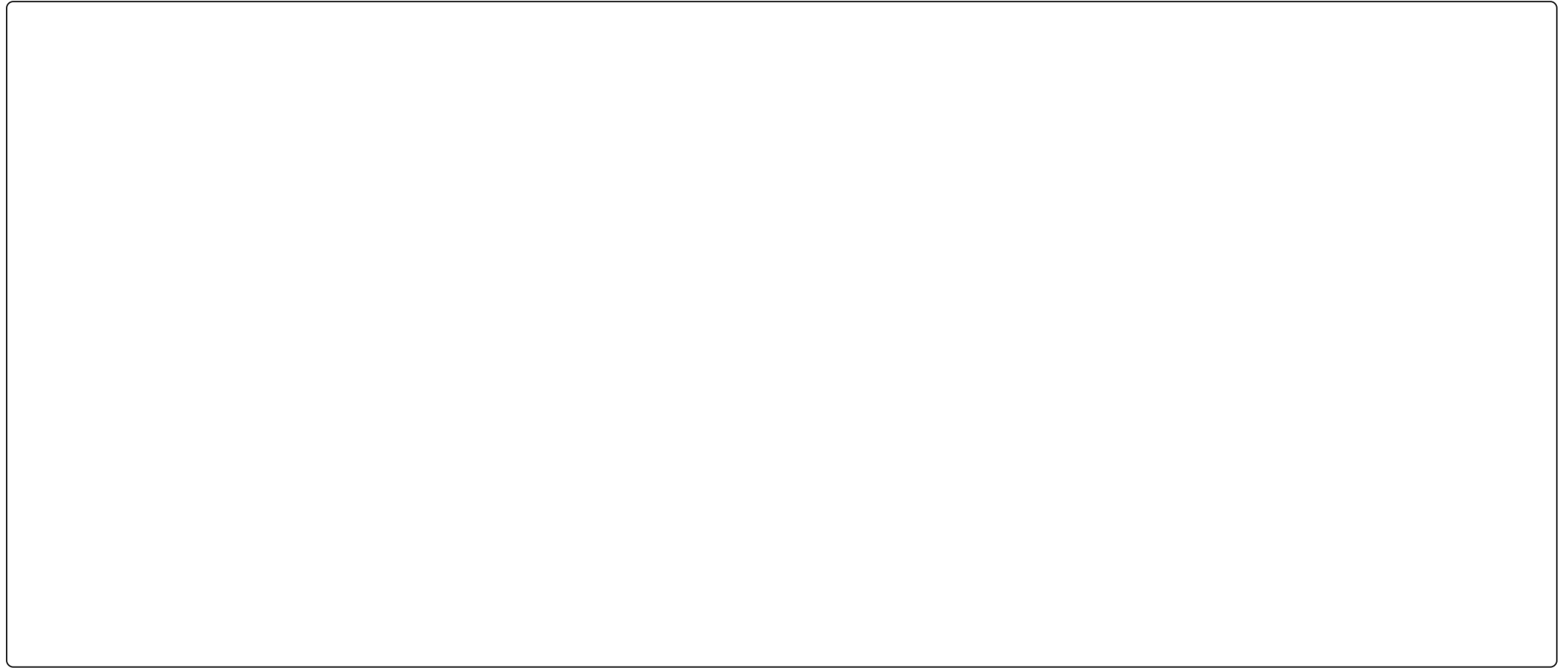
Complexity	Big-O	Example
Constant	$O(1)$	<code>x[i]</code> for a list, <code>k in d</code> for a dict
Logarithmic	$O(\log n)$	binary search in a sorted vector
Linear	$O(n)$	<code>v in x</code> for a list, <code>sum(x)</code>
Quasilinear	$O(n \log n)$	sorting
Quadratic	$O(n^2)$	nested loops over the same input
Exponential	$O(C^n)$	trying every subset

Array



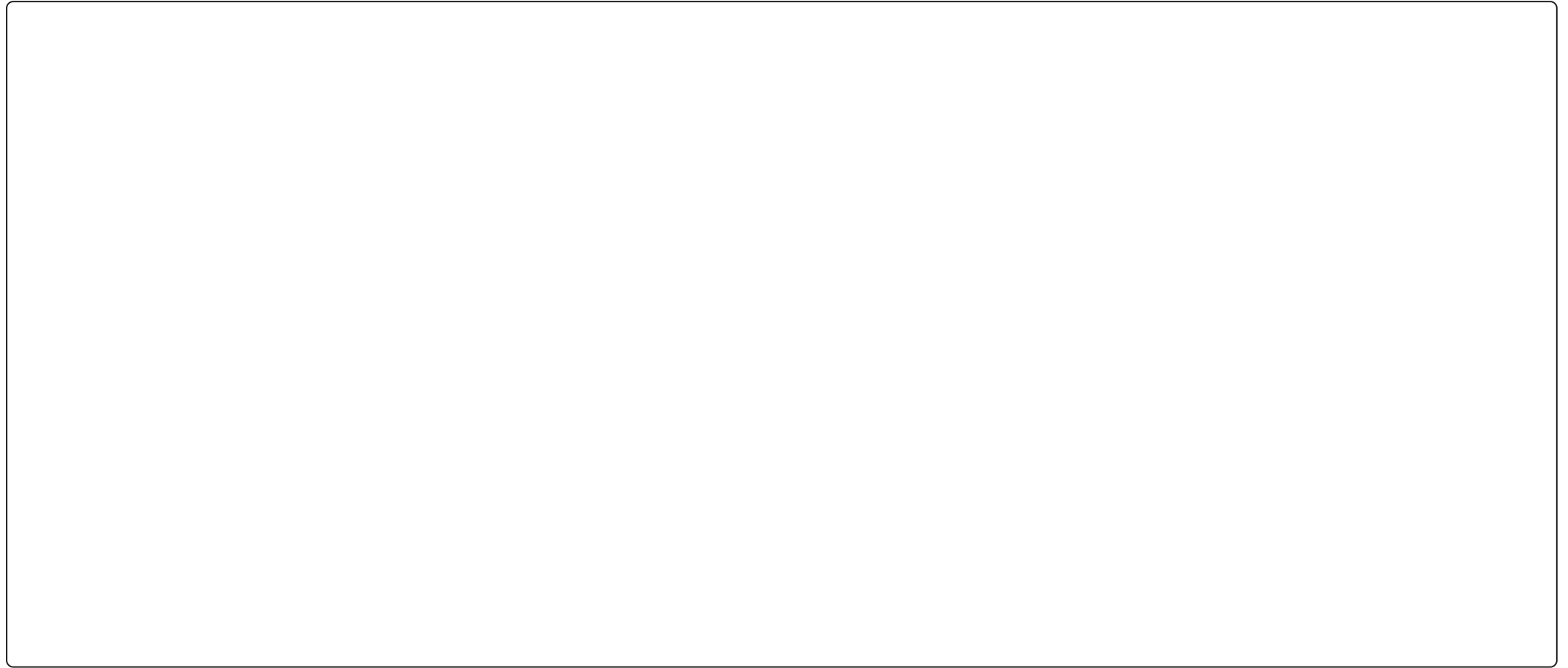
<https://visualgo.net/en/array> - R atomic vectors and NumPy arrays are contiguous blocks of same-typed values. A Python `list` is a contiguous array of *pointers* to objects, which is why it can be heterogeneous and why it uses far more

Linked list



<https://visualgo.net/en/list> - Python's `deque` is a doubly linked list of small arrays. R uses *pairlists* (linked lists) internally

Hash table



<https://visualgo.net/en/hashtable> - Python's `dict` and `set` are hash tables, as are R environments. R's `match()`, `%in%`,
Sta 523 - Fall 2026

Time complexity in Python

Operation	list	dict / set	deque
Copy	$O(n)$	$O(n)$	$O(n)$
Append	$O(1)$	$O(1)$	$O(1)$
Insert	$O(n)$	$O(1)$	$O(n)$
Get item	$O(1)$	$O(1)$	$O(n)$
Set item	$O(1)$	$O(1)$	$O(n)$
Delete item	$O(n)$	$O(1)$	$O(n)$
<code>x in s</code>	$O(n)$	$O(1)$	$O(n)$
<code>pop()</code>	$O(1)$	$O(1)$	$O(1)$
<code>pop(0)</code>	$O(n)$	—	$O(1)$

Average case complexities, from <https://wiki.python.org/moin/TimeComplexity>. Appends are *amortized* $O(1)$ - the list

Growing vectors in R

The same ideas apply in R. `c(x, i)` allocates a *new* vector and copies everything on every iteration ($O(n^2)$ overall), assigning past the end grows in place with over-allocation (since R 3.4), and preallocating avoids the question entirely.

```
1 n = 5e4
```

R

```
1 system.time({  
2   x = c()  
3   for (i in seq_len(n)) x = c(x, i)  
4 })
```

R

user	system	elapsed
1.227	0.025	1.256

```
1 system.time({  
2   x = numeric()  
3   for (i in seq_len(n)) x[length(x) +  
4   })
```

R

user	system	elapsed
0.007	0.000	0.007

```
1 system.time({  
2   x = numeric(n)  
3   for (i in seq_len(n)) x[i] = i  
4 })
```

R

user	system	elapsed
0.001	0.000	0.001

The same applies to lists - `x[[length(x) + 1]] = value` or `vector("list", n)` rather than repeated `c()` or `append()`.

Measuring

Reasoning about complexity tells you what to expect, measuring confirms it.

`time.perf_counter()` and `system.time()` are the simplest tools - here, membership testing in a `list` vs a `set` and a loop vs `%in%`.

```
1 import time
2
3 big_list = list(range(100_000))
4 big_set = set(big_list)
5 queries = range(0, 100_000, 100)
6
7 start = time.perf_counter()
8 res = [q in big_list for q in queries]
9 list_time = time.perf_counter() - start
10
11 start = time.perf_counter()
12 res = [q in big_set for q in queries]
13 set_time = time.perf_counter() - start
14
15 print(f"list: {list_time:.5f}s, set: {set_t
```

list: 0.17761s, set: 0.00214s

```
1 big = sample(1e5)
2 queries = sample(1e5, 1e3)
3
4 system.time({
5   res = vapply(
6     queries, function(q) any(big == q), log
7   )
8 })
```

user	system	elapsed
0.123	0.001	0.125

```
1 system.time({
2   res = queries %in% big
3 })
```

user	system	elapsed
0.005	0.000	0.005

For anything more careful use `timeit` (Python) or `bench::mark()` (R), which repeat the expression many times and

Data structures in R

To tie things together - R's core objects are implemented using the data structures we have just discussed,

- Atomic vectors - array of the given type (`int`, `double`, ...)
- Generic vectors (lists) - array of pointers to R objects (the same layout as a Python `list`)
- Environments - hash table with string keys
- Pairlists - linked list (function arguments, attributes)
- Data frames - a list of equal length vectors (next week)

Exercise 2

For each of the following scenarios, which data structure (in Python and in R) is the most appropriate and why?

- A fixed collection of 100 integers.
- A queue (first in, first out) of customer records.
- A stack (first in, last out) of customer records.
- A count of word occurrences within a document.
- The heights of the bars in a histogram with even bin widths.
- Checking whether each of a million words is in a list of 50 stop words.
- The last 10 lines read from a very large log file.

Comparing R & Python

Container summary

Python	Properties	R
<code>list</code>	ordered, mutable, heterogeneous	<code>list()</code>
<code>tuple</code>	ordered, immutable, heterogeneous	<code>list()</code> or atomic vector (copy on modify)
<code>dict</code>	key lookup, mutable, insertion ordered	named <code>list()</code> , <code>new.env()</code> for hashing
<code>set</code>	unique, unordered, mutable	vector + <code>unique()</code> , <code>union()</code> , <code>%in%</code> , ...
<code>deque</code>	fast at both ends	none (use a list or vector)
<code>range</code>	lazy integer sequence, immutable	<code>seq_len()</code> , <code>:</code> (materialized)
NumPy <code>ndarray</code>	homogeneous, vectorized	atomic vector, matrix, array

Copies & references

Behavior	R	Python
<code>y = x</code> modify <code>y</code> nested structures	<code>y</code> behaves as an independent copy <code>x</code> unchanged (copy on modify) copied at every level	<code>y</code> is another name for the same object <code>x</code> changes too if the object is mutable <code>.copy()</code> is shallow, use <code>copy.deepcopy()</code>
reference semantics lookup by name	environments, R6, external pointers linear scan of <code>names()</code> (hash via <code>new.env()</code>)	every mutable object hash table (<code>dict</code> , <code>set</code>)
growing in a loop	assign past the end or preallocate, avoid <code>c()</code>	<code>.append()</code> is amortized $O(1)$

Takeaways

- Tuples are immutable lists - use them for fixed groups of values and unpack them into names.
- Dictionaries map hashable keys to values with $O(1)$ lookup, R's named lists do the same job for small tables and environments for large ones.
- Sets give $O(1)$ membership and the standard set algebra, R provides the same via `unique()`, `union()`, `intersect()`, `setdiff()`, and `%in%`.
- Only immutable objects can be keys, shallow copies share nested objects, and `[[0] * n] * m` shares its rows - reach for `deepcopy()` when in doubt.
- Know the complexity of the operations you use, and measure when it matters.